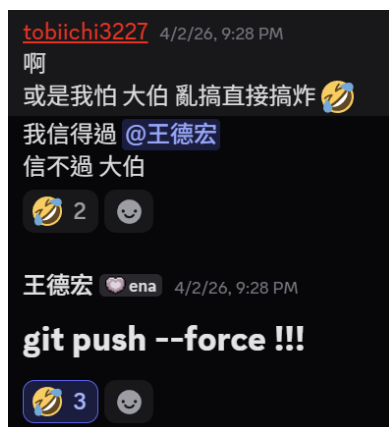


## D. Merge, Rebase, Squash!

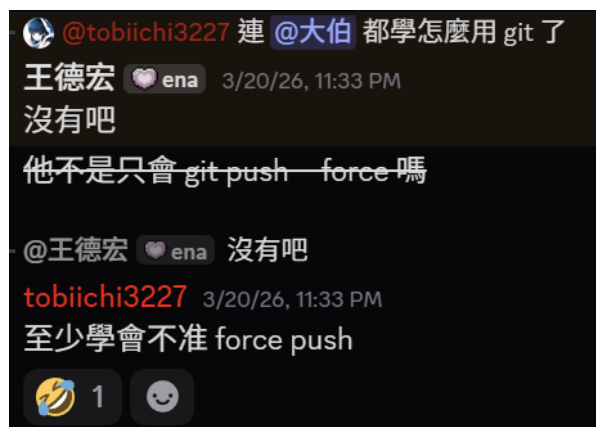
Problem ID: Git

Time Limit: 2.5s

Memory Limit: 512MiB



Force push 1



Force push 2

Figure 1: 不要學大伯 force push (他現在已經不會了)

tobiichi3227 是 TOJ 的維護者。TOJ 的程式碼由  $H$  個互相獨立的 hunk 組成，正式版本位於 branch v2.0。

王德宏身為 TOJ 開發團隊的成員之一，有天想為 TOJ 貢獻一個酷炫的功能，於是很高興地開啟了新的 branch feat/fancy，並在上面依序進行了  $N$  次操作。

王德宏想讓這項功能進到 TOJ 的 branch v2.0，於是請 tobiichi3227 進行審核。tobiichi3227 覺得  $N$  個 commits 太多了，應該先用 squash 將它們整理成恰好  $K$  個 squash commits，再依序 rebase 到 v2.0。

不同的 squash 方式可能產生不同的 conflict 成本。然而，王德宏要去參加國際地理奧林匹亞 (IGGO，全名 International GeoGuesser Olympiad)，已經沒有時間處理了，於是把最小化 conflict 成本的任務交給正在看題目的你。

不用擔心，下面會告訴您 branch rebase squash conflict 的定義。

完整流程合併動畫：[點我下載完整合併 GIF](#)。

## – Branch 與狀態 –

feat/fancy 一開始的所有 hunk 內容皆為 0。令  $S_i[h]$  表示執行完前  $i$  次操作後，第  $h$  個 hunk 的內容，因此：

$$S_0[h] = 0.$$

第  $i$  次操作會把第  $x_i$  個 hunk 改成  $v_i$ ，也就是：

$$S_i[h] = \begin{cases} v_i, & h = x_i, \\ S_{i-1}[h], & h \neq x_i. \end{cases}$$

另一方面，v2.0 上第  $h$  個 hunk 的內容為  $U_h$ 。

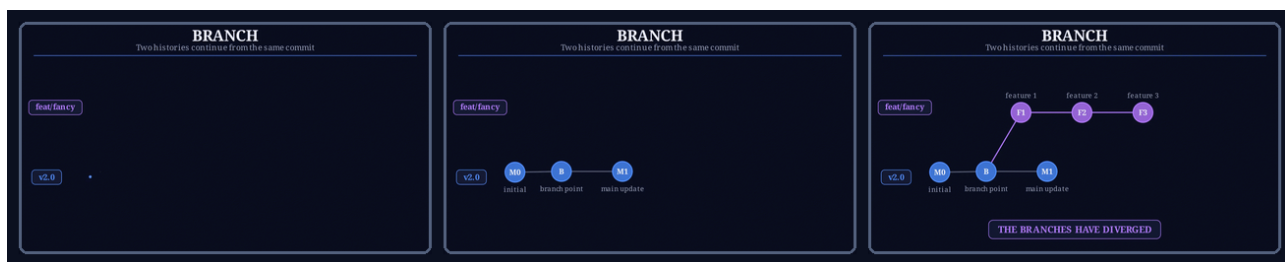


Figure 2: 兩個 branches 從共同位置分開前進

完整動畫：[點我下載 Branch GIF](#)。

## – Squash –

你需要將這  $N$  次操作依序切分成恰好  $K$  個連續的區間（即  $K$  個 squash commits）。

假設其中一個區間包含了第  $L$  到第  $R$  次操作（ $1 \leq L \leq R \leq N$ ），該區間對第  $h$  個 hunk 的實質修改定義為：

- 修改前（舊內容）： $\text{old} = S_{L-1}[h]$
- 修改後（新內容）： $\text{new} = S_R[h]$

若  $\text{old} = \text{new}$ ，則視為該區間對第  $h$  個 hunk 沒有進行修改。

等價地，可以選擇：

$$0 = p_0 < p_1 < \cdots < p_K = N,$$

其中第  $g$  個區間為  $[p_{g-1} + 1, p_g]$ 。

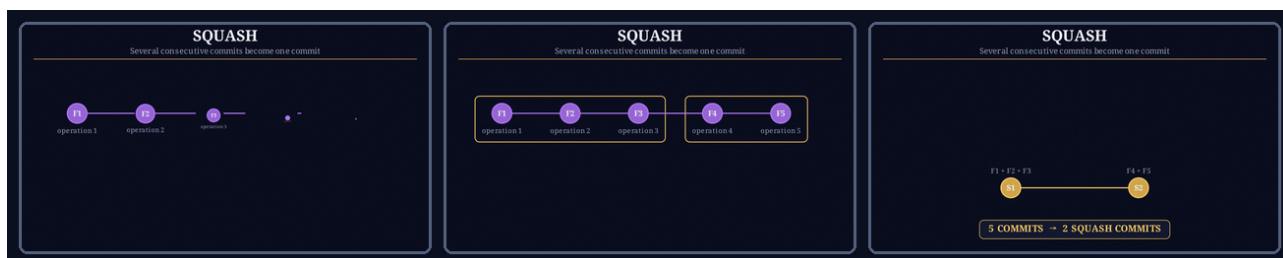


Figure 3: 每個連續區間形成一個 squash commit

完整動畫：[點我下載 Squash GIF](#)。

## – Rebase –

這  $K$  個區間會按照原本順序逐一套用。令  $\text{current}[h]$  表示 v2.0 上第  $h$  個 hunk 目前的內容；一開始：

$$\text{current}[h] = U_h.$$

處理一個區間時，會分別考慮每個 hunk 的 old、new 與  $\text{current}[h]$ 。

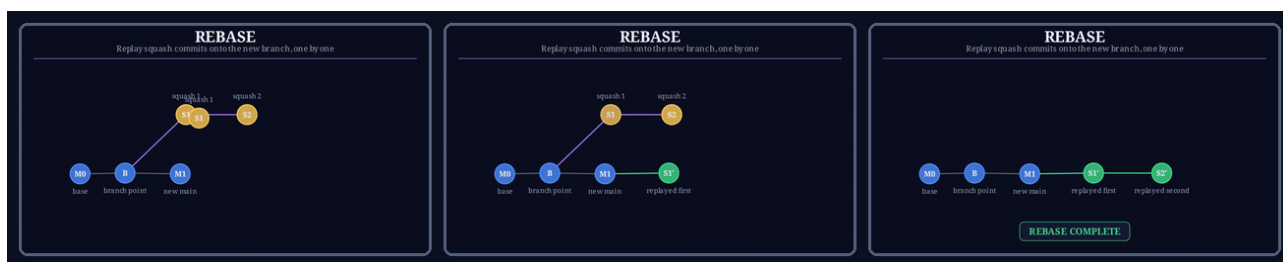


Figure 4: 各 squash commits 依序 replay 到 v2.0

完整動畫：[點我下載 Rebase GIF](#)。

## – Conflict –

若  $old = new$ ，該 hunk 沒有被區間修改， $current[h]$  不變。

否則，當且僅當：

$$current[h] \neq old \quad \text{且} \quad current[h] \neq new$$

時會發生 conflict，並產生  $w_h$  的成本。

不論是否發生 conflict，處理完這項實質修改後皆令：

$$current[h] \leftarrow new.$$

每個區間、每個 hunk 的 conflict 成本分別計算；你的目標是選擇  $K$  個區間，使總成本最小。



Figure 5: 發生 conflict 的兩個 commits 閃紅，解決後轉為綠色

完整動畫：[點我下載 Conflict GIF](#)。

## — 輸入 —

第一行包含三個整數  $N, H, K$ ，分別表示：

- feat/fancy branch 上的 commit 數量；
- hunk 的數量；
- 需要切分出的連續區間數量。

第二行包含  $H$  個整數  $U_h$  ( $1 \leq h \leq H$ )

其中  $U_h$  表示 v2.0 branch 上第  $h$  個 hunk 的最終內容。

第三行包含  $H$  個整數  $w_h$  ( $1 \leq h \leq H$ )

其中  $w_h$  表示解決第  $h$  個 hunk 的 conflict 所需的成本。

接下來  $N$  行，第  $i$  ( $1 \leq i \leq N$ ) 行包含兩個整數  $x_i$  與  $v_i$   
表示第  $i$  個 commit 將第  $x_i$  個 hunk 的內容改成  $v_i$ 。

## — 輸出 —

輸出一個整數，表示最小的 conflict 解決總成本。

## — 輸入限制 —

- $1 \leq N, H \leq 200000$
- $1 \leq K \leq \min(N, 30)$
- $1 \leq U_h \leq 10^9$
- $1 \leq w_h \leq 10^9$
- $1 \leq x_i \leq H$
- $1 \leq v_i \leq 10^9$
- 對於每個 commit  $i : v_i \neq S_{i-1}[x_i]$
- 保證每個 commit 都會實際改變該 hunk 的內容，且 feat/fancy branch 不會將任何 hunk 改回 0。

### — 子任務 —

| 編號 | 分數 | 額外限制             |
|----|----|------------------|
| 1  | 0  | 範例輸入輸出           |
| 2  | 8  | $N, H \leq 15$   |
| 3  | 8  | $K = 1$          |
| 4  | 8  | $K = N$          |
| 5  | 10 | 每個 hunk 最多只被修改一次 |
| 6  | 26 | $N \leq 1000$    |
| 7  | 40 | 無限制              |

### — 範例輸入 —

```
5 2 2
1 4
5 3
1 1
2 2
1 2
2 4
1 3
```

### — 範例輸出 —

```
0
```

### — 範例說明 —

feat/fancy branch 上兩個 hunk 的狀態依序如下：

```
commit 0: (0, 0)
commit 1: (1, 0)
commit 2: (1, 2)
commit 3: (2, 2)
commit 4: (2, 4)
commit 5: (3, 4)
```

可以將 commits 分成：

[1] [2, 3, 4, 5]

第一個 squash commit 將 hunk 1 :

0 -> 1

套用到 v2.0 branch 時，hunk 1 的目前內容已經是 1，因此不會發生 conflict。

第二個 squash commit 的效果為：

hunk 1 : 1 -> 3

hunk 2 : 0 -> 4

此時 hunk 1 的目前內容為 1，可以正常套用。

hunk 2 的目前內容已經是 4，因此也不會發生 conflict。

總成本為 0。

## — 範例輸入 —

```
2 1 1
1
7
1 1
1 2
```

## — 範例輸出 —

```
7
```

## — 範例說明 —

因為  $K = 1$ ，兩個 commits 必須全部 squash 在一起。  
Squash 後的效果為：

```
0 -> 2
```

但 v2.0 branch 上該 hunk 的內容為 1。

目前內容既不是 patch 的舊內容 0，也不是新內容 2，因此發生 conflict，成本為 7。

## — 範例輸入 —

```
2 1 2
1
7
1 1
1 2
```

## — 範例輸出 —

```
0
```

## — 範例說明 —

因為  $K = 2$ ，兩個 commits 分別套用。  
第一個 commit 的效果為：

```
0 -> 1
```

而 v2.0 branch 上目前的內容已經是 1，因此不會發生 conflict。

第二個 commit 的效果為：

1 -> 2

此時目前內容為 1，可以正常套用。

因此總成本為 0。

## — Git 小教室 —

下面會用一個小型 repository 實際重現題目中的 branch、squash、rebase 與 conflict。整份教學都能在**沒有網路**的環境執行：toj.git 是建立在同一台電腦上的本機 remote，不會連線到 GitHub。命令以 Bash 或 Git Bash 為例；程式碼框中的命令不用輸入開頭的提示符號。

### Git 在記錄什麼？

- **working tree**：目前資料夾中實際看到的檔案。
- **staging area**：下一個 commit 準備收錄的內容。
- **commit**：repository 在某個時間點的快照，以及指向前一個 commit 的資訊。
- **branch**：指向某個 commit、並會隨新 commit 前進的名稱，例如 v2.0、feat/fancy。
- **HEAD**：目前 checkout 的 branch 或 commit。
- **remote**：另一個 repository 的別名；remote 可以在網路上，也可以只是本機路徑。

最常使用的命令如下：

|                                 |                                     |
|---------------------------------|-------------------------------------|
| git init                        | 建立 repository                       |
| git status                      | 查看目前 branch、已修改與已 stage 的檔案         |
| git diff                        | 查看尚未 stage 的差異                      |
| git diff --staged               | 查看已 stage、將進入下一個 commit 的差異         |
| git add <file>                  | 將檔案的目前內容放進 staging area             |
| git commit -m "<message>"       | 建立 commit                           |
| git log --graph --oneline --all | 用簡圖查看所有 branch 的歷史                  |
| git branch                      | 列出 branch                           |
| git switch <branch>             | 切換 branch                           |
| git switch -c <branch>          | 建立並切換到新 branch                      |
| git merge <branch>              | 把指定 branch 合併到目前 branch             |
| git rebase <branch>             | 把目前 branch 的 commits 逐一重播到指定 branch |

`git commit -am "<message>"` 是 `git add` 已追蹤檔案再 `git commit` 的簡寫，但**不會加入新建立、尚未追蹤的檔案**。不確定目前狀態時，先執行 `git status` 通常最安全。

## 1. 建立完全離線的 repository

先建立一個新的空資料夾。`toj.git` 模擬遠端的 TOJ repository，`work` 則是實際工作的 repository：

```
mkdir git-classroom
cd git-classroom
git init --bare toj.git
git init -b v2.0 work
cd work
```

bare repository 沒有 working tree，適合當作大家交換 commits 的 remote。接著只替這個練習設定作者資料；`example.invalid` 是保留給範例使用的網域，不需要網路：

```
git config user.name "Git Student"
git config user.email "student@example.invalid"
```

用兩個一行文字檔模擬兩個 hunk，並建立共同的起始 commit：

```
printf "0\n" > hunk-1.txt
printf "0\n" > hunk-2.txt
git add hunk-1.txt hunk-2.txt
git status
git commit -m "base: all hunks are 0"
git tag base
```

base tag 只是替這個 commit 取一個固定名稱，之後進行 squash 時便不必抄寫隨機產生的 commit hash。

## 2. 在 feat/fancy 建立四個 commits

從共同起點建立新 branch：

```
git switch -c feat/fancy
```

依序建立四個 commits。這對應題目中同一個 hunk 可能被多次修改：

```
printf "3\n" > hunk-1.txt
git commit -am "fancy 1: hunk 1, 0 to 3"
```

```
printf "4\n" > hunk-2.txt
git commit -am "fancy 2: hunk 2, 0 to 4"
```

```
printf "5\n" > hunk-1.txt
git commit -am "fancy 3: hunk 1, 3 to 5"
```

```
printf "7\n" > hunk-1.txt
git commit -am "fancy 4: hunk 1, 5 to 7"
```

可以查看目前的四個 feature commits：

```
git log --oneline --decorate
```

將本機 bare repository 命名為 origin，並保存 squash 前的 branch：

```
git remote add origin ../toj.git
git push -u origin feat/fancy
```

-u 會記住本機 feat/fancy 對應到 origin/feat/fancy。這裡的 remote 是 ../toj.git，因此 push 全程只會存取本機檔案。

### 3. 讓 v2.0 同時向前進

切回 v2.0，令它的兩個 hunk 最終為 (5, 4)：

```
git switch v2.0
printf "5\n" > hunk-1.txt
printf "4\n" > hunk-2.txt
git commit -am "v2.0: set final hunk states"
git push -u origin v2.0
git log --graph --oneline --decorate --all
```

現在兩個 branches 從 base 分岔：

```

          fancy 1 -- fancy 2 -- fancy 3 -- fancy 4 (feat/fancy)
        /
base: (0, 0) --
        \
          v2.0: (5, 4) (v2.0)

```

#### 4. Merge 與 Rebase 有什麼不同？

merge 會把另一條 branch 的最終結果合進目前 branch；若兩邊都有新 commits，通常會另外建立一個有兩個 parents 的 merge commit。先建立不影響正式流程的 merge-demo：

```
git switch -c merge-demo v2.0
git merge feat/fancy
```

兩邊都把 hunk-1.txt 從 0 改成不同內容，因此 Git 會回報 conflict。兩邊都把 hunk-2.txt 改成 4，所以該檔案可以自動合併。用以下命令可以查看未解決的檔案：

```
git status
git diff
```

這次只是在比較 merge，所以放棄它並回到 feature branch：

```
git merge --abort
git switch feat/fancy
```

相對地，rebase v2.0 會找出 feat/fancy 與 v2.0 的共同祖先，將 feature commits 暫時取下，再按原順序逐一套用到新版 v2.0 後面。因此 rebase 後的歷史通常較直線；在這個已經分岔、確實需要 replay 的例子中，commit 的 parent 與內容雜湊改變，commit hash 也會改變。

#### 5. 把四個 commits squash 成兩個

題目要求把連續的 commits 分組。這裡示範 [1, 2] [3, 4]，也就是令  $K = 2$ 。在 feat/fancy 上執行互動式 rebase：

```
git rebase -i base
```

編輯器會依照由舊到新的順序列出四行，commit hash 會因電腦與執行時間而不同：

```
pick <hash 1> fancy 1: hunk 1, 0 to 3
pick <hash 2> fancy 2: hunk 2, 0 to 4
pick <hash 3> fancy 3: hunk 1, 3 to 5
pick <hash 4> fancy 4: hunk 1, 5 to 7
```

把每一組中除了第一個以外的 pick 改成 squash：

```
pick    <hash 1> fancy 1: hunk 1, 0 to 3
squash  <hash 2> fancy 2: hunk 2, 0 to 4
pick    <hash 3> fancy 3: hunk 1, 3 to 5
squash  <hash 4> fancy 4: hunk 1, 5 to 7
```

儲存並關閉編輯器；若 Git 再要求編輯合併後的 commit message，確認內容後同樣儲存並關閉。第一個 squash commit 的效果為：

```
hunk 1: 0 -> 3
hunk 2: 0 -> 4
```

第二個 squash commit 的效果為：

```
hunk 1: 3 -> 7
```

中間的 3 -> 5 -> 7 已經縮成 3 -> 7。確認 base 後面現在恰有兩個 commits：

```
git rev-list --count base..feat/fancy
git log --oneline --reverse base..feat/fancy
```

第一個命令應輸出 2。真實 repository 若沒有事先建立 base tag，可以先用 git merge-base v2.0 feat/fancy 找共同祖先，再將輸出的 commit hash 交給 git rebase -i。

## 6. Rebase 並解決 conflict

把這兩個 squash commits 逐一 replay 到 v2.0：

```
git rebase v2.0
```

套用第一個 squash commit 時：

- hunk-1 的 patch 是 0 -> 3，但 v2.0 的 current 是 5，因此發生 conflict。
- hunk-2 的 patch 是 0 -> 4，而 v2.0 的 current 已經是 4，因此不發生 conflict。

Git 會停下來等待處理。查看狀態與 conflict markers：

```
git status
cat hunk-1.txt
cat hunk-2.txt
```

hunk-1.txt 會包含以下幾個部分（左欄的說明文字不在實際檔案中）：

```
目前版本的起點標記 | <<<<<<< HEAD
目前版本的內容      | 5
分隔線              | =====
正在 replay 的內容   | 3
feature 版本的標記   | >>>>>>> fancy 1: hunk 1, 0 to 3
```

<<<<<<<、=====、>>>>>>> 是 Git 加入的標記，不是檔案原本的內容。題目規定解決 conflict 後採用 squash commit 的 new，所以刪除標記並把檔案內容改成 3：

```
printf "3\n" > hunk-1.txt
git add hunk-1.txt
git rebase --continue
```

git add 在這裡代表「這個檔案的 conflict 已處理完成」。若 --continue 開啟 commit message 編輯器，直接確認、儲存並關閉即可。接著第二個 squash commit 的 3 -> 7 可以正常套用，rebase 便會完成。

```
git status
cat hunk-1.txt
cat hunk-2.txt
git rev-list --count v2.0..feat/fancy
git log --graph --oneline --decorate v2.0 feat/fancy
```

最後應得到 `hunk-1.txt = 7`、`hunk-2.txt = 4`，而且 `v2.0` 後面恰有兩個 feature commits。若想放棄整次 rebase 並回到開始前，可以在尚未完成時使用：

```
git rebase --abort
```

不要在不了解目前 commit 的情況下使用 `git rebase --skip`；它會直接丟掉正在 replay 的 commit。

## 7. 為什麼 rebase 後需要 force push？

remote 上仍保存 squash 前的四個 commits，但本機已把它們改寫成另外兩個 commits。普通 push 只允許 fast-forward，因此以下命令應被拒絕：

```
git push origin feat/fancy
```

確認沒有人在 remote 上加入你尚未看過的 commit 後，使用：

```
git push --force-with-lease origin feat/fancy
```

`--force-with-lease` 只會在 remote branch 仍是本機所預期的版本時覆寫它；若別人已經更新 remote，push 會被拒絕。它通常比無條件覆寫的 `--force` 安全，但改寫共享 branch 前仍應先和其他協作者確認。

## 題目模型與真實 Git 的差異

本題刻意把程式碼簡化成互相獨立的 hunks，並用 `old`、`new`、`current` 三個整數定義是否 conflict。真實 Git 會考慮檔案中的多行 context、相鄰修改、檔案新增刪除與重新命名等資訊，不能在所有情況下只靠三個值完整描述。上面的練習把每個 hunk 放在獨立的一行文字檔中，因此能重現本題所描述的三種結果；解題時仍應以題目正式定義為準。